Cyberscope

A TAC Security Company

Audit Report

ILLU Token

January 2026

Network BSC

Address 0x3B926BE474C18196879a2396f5c0d069d0e8d580

Audited by © cyberscope

Analysis

Critical Medium Minor / Informative Pass

Severity	Code	Description	Status
●	ST	Stops Transactions	Passed
●	OTUT	Transfers User's Tokens	Passed
●	ELFM	Exceeds Fees Limit	Passed
●	MT	Mints Tokens	Passed
●	BT	Burns Tokens	Passed
●	BC	Blacklists Addresses	Passed

Diagnostics

Critical Medium Minor / Informative

Severity	Code	Description	Status
●	PAM	Possible Approve Misuse	Unresolved
●	ROF	Redundant Ownership Functionality	Unresolved
●	RFO	Redundant Function Overrides	Unresolved
●	RSML	Redundant SafeMath Library	Unresolved
●	L19	Stable Compiler Version	Unresolved

Table of Contents

Analysis 1
Diagnostics 2
Table of Contents 3
Risk Classification 4
Review 5
Audit Updates 5
Source Files 5
Findings Breakdown 6
PAM - Possible Approve Misuse 7
Description 7
Recommendation 7
ROF - Redundant Ownership Functionality 8
Description 8
Recommendation 8
RFO - Redundant Function Overrides 9
Description 9
Recommendation 10
RSML - Redundant SafeMath Library 11
Description 11
Recommendation 11
L19 - Stable Compiler Version 13
Description 13
Recommendation 13
Functions Analysis 14
Inheritance Graph 15
Flow Graph 16
Summary 17
Disclaimer 18
About Cyberscope 19

Risk Classification

The criticality of findings in Cyberscope's smart contract audits is determined by evaluating multiple variables. The two primary variables are:

1. **Likelihood of Exploitation**: This considers how easily an attack can be executed, including the economic feasibility for an attacker.
2. **Impact of Exploitation**: This assesses the potential consequences of an attack, particularly in terms of the loss of funds or disruption to the contract's functionality.

Based on these variables, findings are categorized into the following severity levels:

1. **Critical**: Indicates a vulnerability that is both highly likely to be exploited and can result in significant fund loss or severe disruption. Immediate action is required to address these issues.
2. **Medium**: Refers to vulnerabilities that are either less likely to be exploited or would have a moderate impact if exploited. These issues should be addressed in due course to ensure overall contract security.
3. **Minor**: Involves vulnerabilities that are unlikely to be exploited and would have a minor impact. These findings should still be considered for resolution to maintain best practices in security.
4. **Informative**: Points out potential improvements or informational notes that do not pose an immediate risk. Addressing these can enhance the overall quality and robustness of the contract.

Severity	Likelihood / Impact of Exploitation
● Critical	Highly Likely / High Impact
● Medium	Less Likely / High Impact or Highly Likely/ Lower Impact
● Minor / Informative	Unlikely / Low to no Impact

Review

Contract Name	ILLUToken
Compiler Version	v0.8.26+commit.8a97fa7a
Optimization	200 runs
Explorer	https://bscscan.com/address/0x3b926be474c18196879a2396f5c0d069d0e8d580
Address	0x3b926be474c18196879a2396f5c0d069d0e8d580
Network	BSC
Symbol	ILLU
Decimals	18
Total Supply	52,000,000

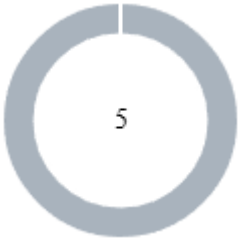
Audit Updates

Initial Audit	28 Jan 2026

Source Files

Filename	SHA256
Illu.sol	61072f1c008b0e5ceb827c8796bddab481e54275d60bcb1f34ef9371cb5779e6

Findings Breakdown



Critical	0
Medium	0
Minor / Informative	5

Severity	Unresolved	Acknowledged	Resolved	Other
Critical	0	0	0	0
Medium	0	0	0	0
Minor / Informative	5	0	0	0

PAM - Possible Approve Misuse

Criticality	Minor / Informative
Location	Illu.sol#L39
Status	Unresolved

Description

The `burnFrom()` function decreases allowances by reading `allowance()` and updating it via `_approve(...)`, instead of using the `_spendAllowance(...)` mechanism. This deviates from the most common implementations' allowance semantics where type(uint256).max is treated as an infinite approval and is not decremented. Consequently, `burnFrom()` will reduce the allowance even under infinite approvals and may introduce an unnecessary allowance state update.

```shell
function burnFrom(address account, uint256 amount) public {
        uint256 decreasedAllowance = allowance(account,
msg.sender).sub(amount, "ERC20: burn amount exceeds
allowance");
        _approve(account, msg.sender, decreasedAllowance);
        _burn(account, amount);

    }
```

Recommendation

The team is advised to align the allowance handling logic with the standard OpenZeppelin implementation to improve consistency and avoid unnecessary state updates.

ROF - Redundant Ownership Functionality

Criticality	Minor / Informative
Location	Illu.sol#L
Status	Unresolved

Description

The contract implements functionality to define an owner. This functionality is normally implemented in contracts that need some form of authority and access control. However, excluding the transferOwnership there is no function in the contract that needs to be called only by the owner. Therefore the Ownable functionality is redundant.

```shell
function transferOwnership(address newOwner) public
override onlyOwner {
```

Recommendation

It is recommended to remove the ownable functionality to increase code optimization and readability.

RFO - Redundant Function Overrides

Criticality	Minor / Informative
Location	Illu.sol#L16,21,26,45
Status	Unresolved

Description

The contract overrides `transfer`, `approve`, `transferFrom`, and `transferOwnership` to add additional require checks that the involved addresses are not address(0). However, each function subsequently calls the corresponding OpenZeppelin implementation via `super.transfer` `super.approve` `super.transferFrom` `_transferOwnership`, which already performs the same zero-address validations internally. As a result, these extra checks do not provide additional security guarantees and primarily increase code size/complexity.

```rust
function transfer(address recipient, uint256 amount) public
override returns (bool) {
        require(recipient != address(0), "ERC20: transfer
to the zero address");
        return super.transfer(recipient, amount);
    }

function approve(address spender, uint256 amount) public
override returns (bool) {
        require(spender != address(0), "ERC20: approve to
the zero address");
        return super.approve(spender, amount);
    }

function transferFrom(address sender, address recipient,
uint256 amount) public override returns (bool) {
        require(sender != address(0), "ERC20: transfer from
```

```
the zero address");
        require(recipient != address(0), "ERC20: transfer
to the zero address");
        return super.transferFrom(sender, recipient,
amount);
    }
function transferOwnership(address newOwner) public
override onlyOwner {
        require(newOwner != address(0), "Ownable: new owner
is the zero address");
        _transferOwnership(newOwner);

    }
```

Recommendation

The team is advised to remove redundant code where possible to reduce complexity.

RSML - Redundant SafeMath Library

Criticality	Minor / Informative
Location	SignedMath.sol SafeMath.sol Illu.sol
Status	Unresolved

Description

SafeMath is a popular Solidity libraries that provides a set of functions for performing common arithmetic operations in a way that is resistant to integer overflows and underflows.

Starting with Solidity versions that are greater than or equal to 0.8.0, the arithmetic operations revert to underflow and overflow. As a result, the native functionality of the Solidity operations replaces the SafeMath and SignedMath librarys. Hence, the usage of the SafeMath library and SignedMath library adds complexity, overhead and increases gas consumption unnecessarily in cases where the explanatory error message is not used. The SafeMath and SignedMath libraries are completely identical, resulting in duplicated code.

```shell
library SafeMath {...}
```

Recommendation

The team is advised to remove the SafeMath library in cases where the revert error message is not used. Since the version of the contract is greater than `0.8.0` then the pure Solidity arithmetic operations produce the same result.

If the previous functionality is required, then the contract could exploit the `unchecked { ... }` statement.

Read more about the breaking change on https://docs.soliditylang.org/en/stable/080-breaking-changes.html#solidity-v0-8-0-breaking-changes.

L19 - Stable Compiler Version

Criticality	Minor / Informative
Location	Illu.sol#L2
Status	Unresolved

Description

The `^` symbol indicates that any version of Solidity that is compatible with the specified version (i.e., any version that is a higher minor or patch version) can be used to compile the contract. The version lock is a mechanism that allows the author to specify a minimum version of the Solidity compiler that must be used to compile the contract code. This is useful because it ensures that the contract will be compiled using a version of the compiler that is known to be compatible with the code.

```shell
pragma solidity ^0.8.26;
```

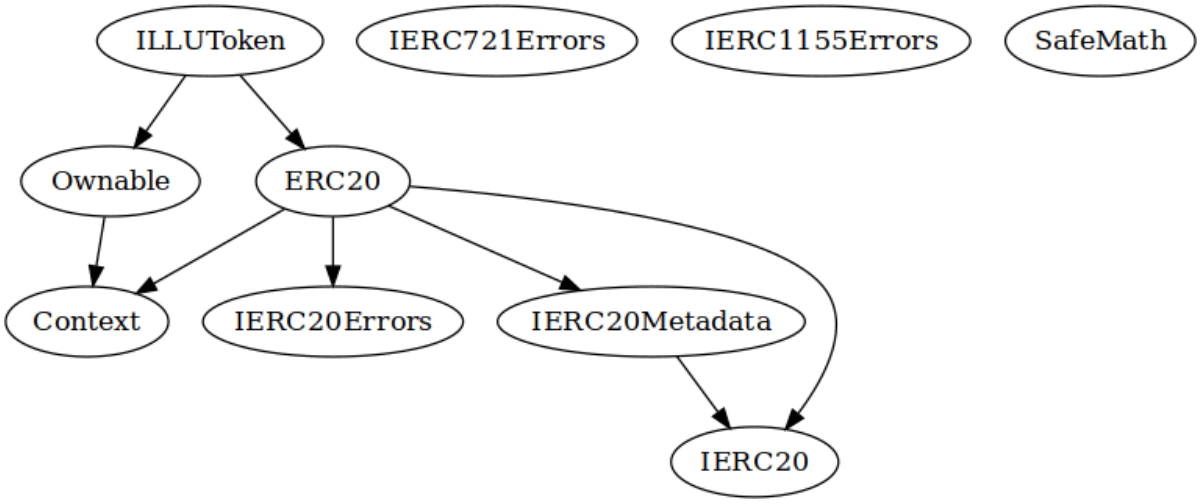
Recommendation

The team is advised to lock the pragma to ensure the stability of the codebase. The locked pragma version ensures that the contract will not be deployed with an unexpected version. An unexpected version may produce vulnerabilities and undiscovered bugs. The compiler should be configured to the lowest version that provides all the required functionality for the codebase. As a result, the project will be compiled in a well-tested LTS (Long Term Support) environment.

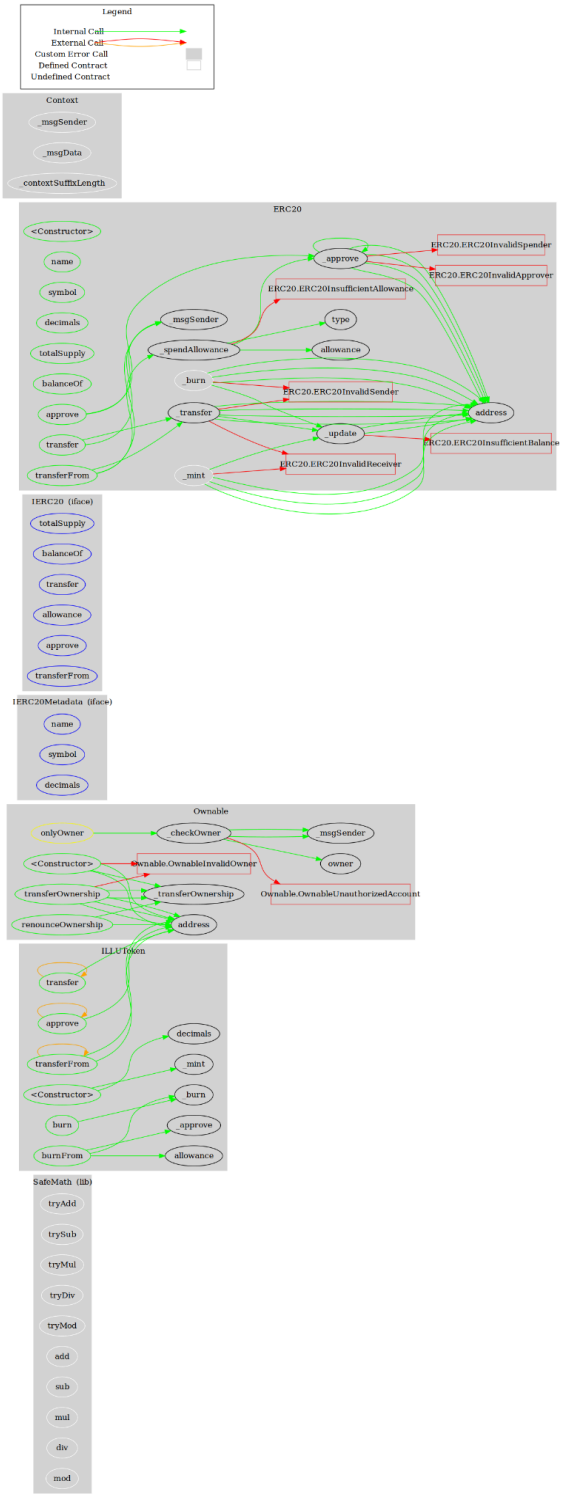
Functions Analysis

Contract	Type	Bases		
	Function Name	**Visibility**	**Mutability**	**Modifiers**
ILLUToken	Implementation	ERC20, Ownable		
		Public	✓	ERC20 Ownable
	transfer	Public	✓	-
	approve	Public	✓	-
	transferFrom	Public	✓	-
	burn	Public	✓	-
	burnFrom	Public	✓	-
	transferOwnership	Public	✓	onlyOwner

Inheritance Graph



Flow Graph


Legend
Internal Call
External Call
Custom Error Call
Defined Contract
Undefined Contract
Context
_msgSender
_msgData
_contextSuffixLength
ERC20
<Constructor>
name
symbol
decimals
totalSupply
balanceOf
approve
transfer
transferFrom
_msgSender
_spendAllowance
_burn
transfer
_mint
approve
_update
type
allowance
address
ERC20.ERC20InvalidSpender
ERC20.ERC20InvalidApprover
ERC20.ERC20InsufficientAllowance
ERC20.ERC20InvalidSender
ERC20.ERC20InsufficientBalance
ERC20.ERC20InvalidReceiver
IERC20 (iface)
totalSupply
balanceOf
transfer
allowance
approve
transferFrom
IERC20Metadata (iface)
name
symbol
decimals
Ownable
onlyOwner
<Constructor>
transferOwnership
renounceOwnership
_checkOwner
_msgSender
owner
Ownable.OwnableInvalidOwner
transferOwnership
Ownable.OwnableUnauthorizedAccount
address
ILLUToken
transfer
approve
transferFrom
<Constructor>
burn
burnFrom
decimals
_mint
_burn
_approve
allowance
SafeMath (lib)
tryAdd
trySub
tryMul
tryDiv
tryMod
add
sub
mul
div
mod

Summary

I Love & Like Universe Token contract implements a token mechanism. This audit investigates security issues, business logic concerns and potential improvements. I Love & Like Universe Token is an interesting project that has a friendly and growing community. The Smart Contract analysis reported no compiler error or critical issues.

Disclaimer

The information provided in this report does not constitute investment, financial or trading advice and you should not treat any of the document's content as such. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes nor may copies be delivered to any other person other than the Company without Cyberscope's prior written consent. This report is not nor should be considered an "endorsement" or "disapproval" of any particular project or team. This report is not nor should be regarded as an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Cyberscope to perform a security assessment. This document does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors' business, business model or legal compliance. This report should not be used in any way to make decisions around investment or involvement with any particular project. This report represents an extensive assessment process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk Cyberscope's position is that each company and individual are responsible for their own due diligence and continuous security Cyberscope's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies and in no way claims any guarantee of security or functionality of the technology we agree to analyze. The assessment services provided by Cyberscope are subject to dependencies and are under continuing development. You agree that your access and/or use including but not limited to any services reports and materials will be at your sole risk on an as-is where-is and as-available basis Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives false negatives and other unpredictable results. The services may access and depend upon multiple layers of third parties.

About Cyberscope

Cyberscope is a TAC blockchain cybersecurity company that was founded with the vision to make web3.0 a safer place for investors and developers. Since its launch, it has worked with thousands of projects and is estimated to have secured tens of millions of investors' funds.

Cyberscope is one of the leading smart contract audit firms in the crypto space and has built a high-profile network of clients and partners.

A **TAC Security** Company

The Cyberscope team

cyberscope.io